

Trabajo Práctico N° 4

Administración y Seguridad

Objetivo: Aplicar los conceptos de Control de Acceso Basado en Roles (RBAC), seguridad de datos mediante Vistas, Criptografía (Simétrica y Hashing) y gestión del ciclo de vida de usuarios en un entorno de base de datos relacional.

Ejercicio inicial: Configuración del Servidor (El rol del DBA)

En la clase anterior, el servidor fue limitado intencionalmente a 1 solo hilo de procesamiento para estudiar los planes de ejecución. Como nuevos administradores del sistema, su primera tarea es liberar la potencia del servidor habilitando el paralelismo para que las tareas ejecutadas sobre los 5 millones de registros sean rápidas.

Tarea: Ejecuta el siguiente comando administrativo para restablecer el valor por defecto (2 workers) a nivel global del sistema, y luego reinicia la configuración:

```
-- 1. Alteramos la configuración del sistema (requiere ser superusuario)
```

```
ALTER SYSTEM SET max_parallel_workers_per_gather = 2;
```

```
-- 2. Recargamos la configuración sin apagar la base de datos
```

```
SELECT pg_reload_conf();
```

Verifica que el cambio fue exitoso ejecutando:

```
SHOW max_parallel_workers_per_gather;
```

Parte 1: El Perfil Gerencial (RBAC y Vistas de Reporte)

El directorio de la empresa necesita acceso a la base de datos para tomar decisiones estratégicas. Sin embargo, por políticas de seguridad, los directivos no deben ver los datos transaccionales crudos (para evitar que vean datos personales de clientes individuales), sino únicamente reportes agregados.

Instrucciones:

- Crea un rol llamado rol_gerente.
- Crea un usuario llamado gerente_ana con la contraseña 'Directorio2026' y asígnale el rol correspondiente.

Como DBA (postgres), crea las siguientes tres vistas de reporte:

- v_reporte_ingresos_sucursal: Debe mostrar el nombre de la sucursal y la suma total de los importes (SUM(importe)) de los pedidos realizados en esa sucursal.

- `v_capital_invertido_categoria`: Debe mostrar la categoría de los artículos y el capital total invertido en stock para esa categoría (es decir, la suma de `stock * precio_unitario`).
- `v_metricas_pedidos`: Debe mostrar el estado de los pedidos y la cantidad total de pedidos (`COUNT(*)`) que se encuentran en cada estado.

Otorga al rol `rol_gerente` permisos de lectura (`SELECT`) únicamente sobre estas tres vistas.

Inicia sesión como `gerente_ana` (o usa `SET ROLE gerente_ana;`) e intenta hacer un `SELECT` a la tabla `pedidos` y luego a la vista `v_reporte_ingresos_sucursal`. Deja constancia de los comandos en tu archivo SQL.

Responde: ¿Por qué la creación de Vistas, combinada con RBAC, se considera una herramienta de "Seguridad" y no solo una herramienta de "Comodidad para el programador"? Justifique utilizando el Principio de Privilegio Mínimo.

Parte 2: Criptografía para Datos Sensibles

La empresa quiere empezar a guardar un teléfono de contacto de emergencia de sus clientes, pero este dato es altamente confidencial. Además, los clientes ahora tendrán un PIN numérico de 4 dígitos para confirmar la entrega de sus pedidos.

Modifica la tabla `clientes` (o crea otra tabla si prefieres) para agregar dos columnas:

- `telefono_emergencia_cifrado` (Tipo de dato `BYTEA`)
- `pin_entrega_hash` (Tipo de dato `TEXT`)

Instrucciones:

- I. Inserta o actualiza un cliente (ej. `id_cliente = 1`) guardando el teléfono '555-0199' de forma recuperable (simétrico) usando la clave 'secreto_empresa', y el PIN '4092' de forma no recuperable (hash).
- II. Escribe la consulta SQL exacta que utilizaría el sistema cuando el repartidor intenta verificar si el cliente ingresó el PIN correcto (debe devolver `true` o `false` al comparar con '4092'). *Ver nota a continuación.*
- III. Escribe la consulta SQL para recuperar el teléfono de emergencia en texto plano para el `id_cliente = 1`.

Responde: Si un empleado desleal con permisos de lectura general (`SELECT * FROM clientes`) hace una copia de la base de datos y se la lleva a su casa:

- a) ¿Podrá llamar a los teléfonos de emergencia? ¿Por qué?
- b) ¿Podrá saber qué PIN usa cada cliente? ¿Por qué?

NOTA: Debemos realizar la consulta que compara el hash generado originalmente (con el pin original) con un hash generado con el nuevo pin ingresado. Pero el hash inicial lo generamos con un 'salt' aleatorio. El hashing no usa una "clave maestra" de la empresa

como el cifrado simétrico. Cuando insertamos el PIN original, hicimos esto:

```
crypt('4092', gen_salt('bf'))
```

Ese `gen_salt('bf')` generó caracteres aleatorios (una "semilla" o "salt") y los mezcló con el PIN '4092'. El resultado que se guardó en la columna `pin_entrega_hash` es una cadena larguísima que se ve más o menos así:

```
$2a$06$R.D/E218h91j...<resto del hash>
```

Esa cadena "mágica" contiene tres cosas:

- A. El algoritmo usado (\$2a\$ = blowfish/bcrypt).
- B. El salt aleatorio que se generó ese día (\$06\$R.D/E218h91j...).
- C. El resultado matemático final.

¿Cómo verificamos con `crypt` si el nuevo intento es correcto?

Si el repartidor tipea '4092', la función `crypt` necesita saber con qué "salt" aleatorio se mezcló la primera vez para poder repetir el proceso matemático.

En la función `crypt` de PostgreSQL le podemos pasar el hash viejo completo (la columna) como segundo parámetro y obtendrá de ahí el salt usado originalmente:

```
SELECT (pin_entrega_hash = crypt('pin_ingresado', pin_entrega_hash))
```

Lo que hace `crypt` internamente en esa línea es:

1. Lee la columna `pin_entrega_hash`.
2. Le extrae el salt aleatorio (\$06\$R.D/E218h91j...).
3. Toma el 'pin_ingresado' y lo encripta usando ese mismo salt.
4. Devuelve el nuevo hash generado.

Parte 3: Seguridad Operativa - Limitación de Updates

El equipo de ventas ha estado cometiendo errores al actualizar pedidos. A veces modifican la fecha o el importe sin querer.

Instrucciones:

- Revoca todos los permisos que tenga el rol `rol_vendedor` sobre la tabla `pedidos` (si los tuviera de prácticas anteriores).
- Otorga al rol `rol_vendedor` permisos para insertar (`INSERT`) y leer (`SELECT`) en la tabla `pedidos`.
- Otorga permiso de actualización (`UPDATE`), pero exclusivamente para la columna `estado`. No deben poder modificar ni el importe, ni el artículo, ni la cantidad.

Responde: En el archivo `.sql` inicial, la tabla `pedidos` tiene un constraint: `CONSTRAINT pedidos_importe_check CHECK (importe >= 0)`. ¿Qué diferencia hay entre este control de

CHECK y el control de acceso de otorgar permiso de UPDATE solo a la columna estado?
Explica qué protege cada uno.

Parte 4: Ciclo de vida y Baja Segura

Debido a una reestructuración del directorio, la gerente Ana deja la empresa. Necesitamos dar de baja su acceso asegurando que la base de datos quede consistente.

Escribe la secuencia exacta de comandos SQL (en el orden correcto) para eliminar por completo al usuario gerente_ana del sistema.

Asegúrate de no borrar el rol rol_gerente ni las vistas, ya que el mes que viene entrará un nuevo gerente y se le asignará ese mismo rol.

Responde: Si intentamos hacer un DROP USER gerente_ana; directamente sin revocar sus permisos previamente, PostgreSQL lanza un error. ¿Por qué el motor de base de datos tiene este comportamiento defensivo? ¿Qué pasaría con la matriz de acceso (DAC) si el motor simplemente borrara al usuario ignorando sus relaciones?